LAB REPORT (20%)

Coursework Type	Lab Report
Weightage	20% out of 100%
Hand-out Date	Week 2
Due Date	Submission: Week 10

Learning Outcomes Assessed:

CLO 2	Display programming skills in web and mobile hybrid application development. [P3, PLO3]
CLO 3	Display the ability to construct and deploy a working application using HTML, CSS, JavaScript and ASP.net (C#) programming language. [P3, PLO3]

Student's Declaration:

I declare that:

- ***I understand what is meant by plagiarism***
- ***This assignment is all of my own work and I have acknowledged any use of the published or unpublished works of other people.***
- ***I hold a copy of this assignment which I can produce if the original is lost or damaged***

No.	Name	Student ID	Signature	Section
1.	Lee Gui Ling	I24027926	*ling*	1|1

DATE: 31/3/2026

PENALTY FOR LATE SUBMISSION:

- 1-2 days after the deadline - deduction of 25% of the marks obtained for the assignments
- 3 days or more after the deadline - deduction of 50% of the marks obtained for assignments

LAB REPORT-INDIVIDUAL

Scenario

You are a mobile app developer tasked with creating a simple, creative, and visually appealing mobile application for a client. The client could be an individual, a small business, or a personal brand owner. Your goal is to design and develop a basic mobile app that introduces the client's brand, service, hobby, or interest to the world.

The main focus of this project is UI design, navigation, and layout, rather than complex coding or database functionality. You may include simple interactions (e.g., button clicks or image taps) to enhance the user experience.

Task

1. Choose one topic from the list below that interests you.

2. Then, design and develop a beginner-friendly mobile app using .NET MAUI based on the topic you selected.

#	Topic	Example Pages / Features
1	Personal Brand / Portfolio	Home, About Me, Gallery/Portfolio
2	Travel Guide	Home, Destinations, Travel Tips
3	Café Menu	Home, Menu, Offers/Deals
4	Pet Adoption	Home, Pet List, Tips for Adoption
5	Book Library / Reading App	Home, Book List, Author Info
6	Event Planner	Home, Upcoming Events, Tips/Ideas
7	Fitness / Workout Guide	Home, Workouts, Nutrition Tips
8	Recipe / Cooking	Home, Recipes, Cooking Tips
9	Art / Creative Gallery	Home, Gallery, Inspiration/Blog
10	Music / Playlist	Home, Playlist, Artist Info
11	Movie / TV Show Recommendation	Home, Movie List, Reviews/Tips
12	Study / Revision	Home, Topics, Tips/Guidelines
13	Weather App (UI Only)	Home, City List, Weather Tips
14	Hobby / Collection	Home, Collection Gallery, About Hobby
15	Local Business / Services	Home, Services Offered, Contact Info

2. Requirements

- Home Page: Display app name, tagline, and a representative image.
- Secondary Pages: Include at least 2 additional pages based on your topic (e.g., About, Gallery, Tips, Menu, etc.).
- Navigation: Use TabBar, Shell, or any simple navigation structure to move between pages.
- Design & Layout: Apply colors, fonts, spacing, and images to make the app clean, visually appealing, and user-friendly.
- Optional Features (Bonus): Button click effects, image tap interactions, or simple animations.

Rubric Lab Report

Criteria	Excellent (A / 85–100%)	Good (B / 70–84%)	Satisfactory (C / 55–69%)	Needs Improvement (D / 40–54%)	Fail (<40%)	Marks
1. Home Page Design (15%)	App name, tagline, image, and welcome message are **clearly presented**, visually appealing, and well-aligned	App name, tagline, and image included; minor layout or alignment issues	App name and tagline present; image is small or misaligned; layout somewhat cluttered	App page missing some elements; poor alignment; text or images not clear	Home page missing or very poorly designed	
2. Secondary Pages / Content (20%)	At least **2 pages** (e.g., About, Gallery, Tips) included; content is relevant, well-structured, and visually attractive	2 pages included; content mostly relevant; minor layout issues	2 pages included; content limited or partially irrelevant	Only 1 page included; content poorly structured	Secondary pages missing	
3. Navigation / Usability (15%)	Navigation is smooth, intuitive, and works correctly on all pages	Navigation mostly works; minor issues with navigation between pages	Navigation works on some pages; inconsistent or confusing	Navigation is poorly implemented; difficult to move between pages	No navigation or completely broken	
4. Visual Design & Creativity (25%)	Attractive color scheme, fonts, spacing, images; demonstrates creativity and originality; layout professional	Good color scheme and fonts; layout clean; some creativity	Acceptable design; layout simple; minimal creativity	Poor design choices; layout messy; little to no creativity	No attention to design or layout	
5. Basic Interaction / UI Features (10%)	Buttons, images, or simple animations included and work correctly; adds interactivity to app	Some buttons or image taps work; minor issues	Minimal interaction included; limited functionality	Attempted interaction but not working	No interactive elements	
6. Overall Presentation & Completeness (15%)	App is complete, runs without errors, all required features implemented; polished and professional	App mostly complete; minor missing features or errors	App partially complete; several features missing; runs with minor issues	App incomplete; many missing features; unstable	App not submitted or non-functional	

Submission Guidelines:

2. Other information associated to the report as follow:
    a. Individual work.
    b. You are required to prepare a minimum of 10 pages.
    c. Arrange your documents as following:
        i. Cover Page (First page of this assignment)
        ii. Assignment Rubric
        iii. Table of contents
        iv. References. *You may include any necessary diagrams or pictures. ***References should be cited properly.***
    d. Report Format
        i. The report size should be A4 paper size.
        ii. All headings should be capitalized, bold and left-aligned.
        iii. All paragraphs must be fully justified with 1.5-line spacing.
        iv. Include the header and footer section for each page. The header and footer must be in the size of 10, grey colored and framed.
            • Footer: Page No
        v. Include citation and references in the report.

    e. Please read the following statement properly to avoid penalty:
    Report must be submitted (by the leader/representative only) latest by **(Week 10).**
    **Check the Canvas regularly, frequently and consistently to keep up with any new updates.

Plagiarism & Copying

PLAGIARISM AND COPYING ARE STRICTLY PROHIBITED AND CONSIDERED A SERIOUS OFFENCE. THE FACULTY HAS FULL RIGHTS TO TAKE DISCIPLINARY ACTIONS AGAINST ANY STUDENT WHO PLAGIARISE OR COPY THE WORK OF OTHERS. AN ACCEPTABLE 'TURN IT IN' REPORT MUST BE INCLUDED AS APPENDIX FOR ALL SUBMITTED ASSIGNMENTS.

The faculty reserves the right to use any means to identify plagiarism and copying. If you reference the work of other authors, you must ensure that you properly reference them to avoid the charge of plagiarism.

Copying the work of others (current students, previous students, or the works of others) automatically earns you a Fail grade. If two or more students copied from each other, both students (regardless of who copies from whom) will earn a Fail grade

Table of Contents

1.0 Introduction

This lab report will outline the design, development, and implementation of a cross-platform mobile application named TravelGuide, created using .NET MAUI (Multi-platform App UI) and C# programming languages. This application will be a travel assistant, offering users recommendations, information, and guidance on various travel destinations.

The specific purpose of this project is to demonstrate expertise in mobile application development, covering user interface design, Model-View-ViewModel architecture, data binding, navigation, and interactive animations. This application, named TravelGuide, has been created to operate on both Android and iOS platforms, leveraging a single shared codebase. This, in turn, showcases the power of cross-platform application development.

This lab report will be divided into sections to:

(i) outline each of the screens within the application
(ii) outline the functionality of the application
(iii) outline the technical implementation of specific application features
(iv) outline the application itself, using screenshots captured from the Android emulator, Pixel 7, API 36.

2.0 Application Overview

TravelGuide is organized around three primary tabs accessible from the persistent bottom navigation bar:

- Home
- Destinations
- Tips

A fourth page, named the Destination Detail Page, is accessible as a route-based sub-page of either the Home or Destinations tab. The application uses a color palette that is warm and earthy, with terracotta orange, cream whites, and various shades of grey to evoke feelings of warmth and excitement associated with travel.

The application contains the following pages:

- **Home Page**: landing screen with hero banner, search bar, category pills, and popular destinations.
- **Destinations Page:** scrollable list of destination cards with images, descriptions, ratings, and a favorite/heart toggle.
- **Travel Tips Page:** scrollable list of practical travel advice cards with emoji icons and a motivational quote banner.
- **Destination Detail Page:** full-screen detail view for Bali and Kyoto, accessed via navigation from both the Home and Destinations tabs.

3.0 Technology Stack

The application was developed using the following technologies and tools:

- **.NET MAUI (Multi-platform App UI):** the core cross-platform framework providing a single C# and XAML codebase deployable to Android, iOS, macOS, and Windows.
- **C#:** the primary programming language used for all business logic, ViewModels, and code-behind files.
- **XAML (eXtensible Application Markup Language):** used to define the user interface layouts for all pages.
- **CommunityToolkit.Mvvm:** A NuGet package that provides source-generated MVVM features including ObservableObject, ObservableProperty, and RelayCommand, reducing boilerplate code significantly.
- **Android Emulator (Pixel 7, API 36):** used for testing and capturing screenshots of the running application.

4.0 Home Page

The Home Page is a screen in TravelGuide, and this is where a user will first enter the application. The Home Page is designed to engage a user right away with a full-width hero image banner and a call to action. The Home Page is constructed in a file called HomePage.xaml and a code file called HomeViewModel.cs.

4.1 UI Layout and Components

The Home Page is composed of the following visible sections from top to bottom:

- **Hero Banner** - a full-width image of hot air balloons over Cappadocia, overlaid with the text "DISCOVER" (small caps), "Explore the World's Wonders" (large white bold), and an orange "Start Exploring ->" button.
- **Search Bar** - a rounded white card beneath the hero image containing a magnifying glass icon and placeholder text "Search destinations...". It responds to hover (scale animation) and tap interactions.
- **Category Pills** - a horizontal row of four rounded pill buttons: Beach, Mountains, Culture, and Adventure. The currently selected pill is highlighted in terracotta orange; all others appear in a soft peach/cream tone.
- **Popular Destinations Section** — a header row with "Popular Destinations" label and an orange "See All" link, followed by a horizontal row of small destination mini-cards showing a photo, destination name, and country.

Below is the Home Page screenshot showing the default state with Beach selected and Bali displayed as the popular destination:

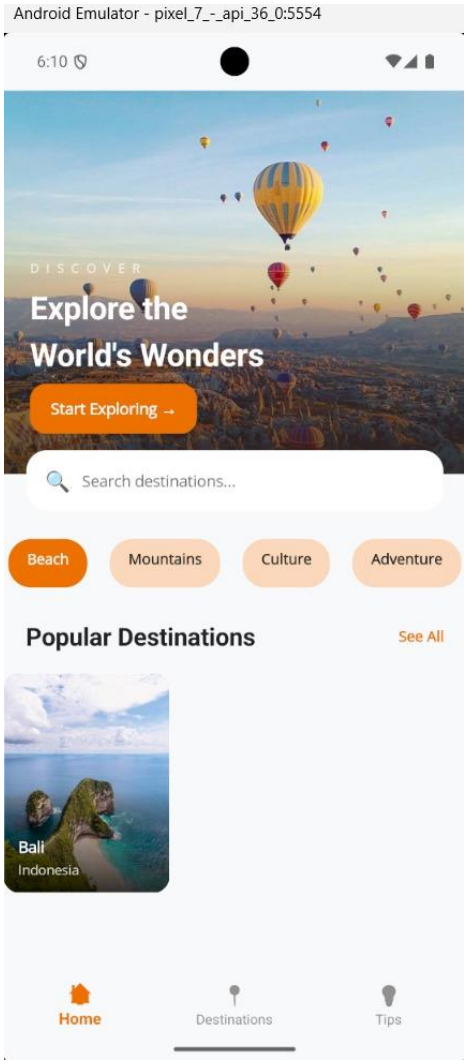

Figure 1: Home Page (Default – Beach selected)

4.2 Category Pill Feature

The pill selection for categories is an important interactive element on the Home Page. When the user taps on any pill within the category system, the SelectCategoryCommand is executed. This command cycles through all the CategoryItem objects within the Categories collection and sets the IsSelected property to true for the clicked object. The XAML binding is reactive with the background color.

The three screenshots below shows the Mountains, Culture, and Adventure pill states, each updating the popular destination card accordingly:

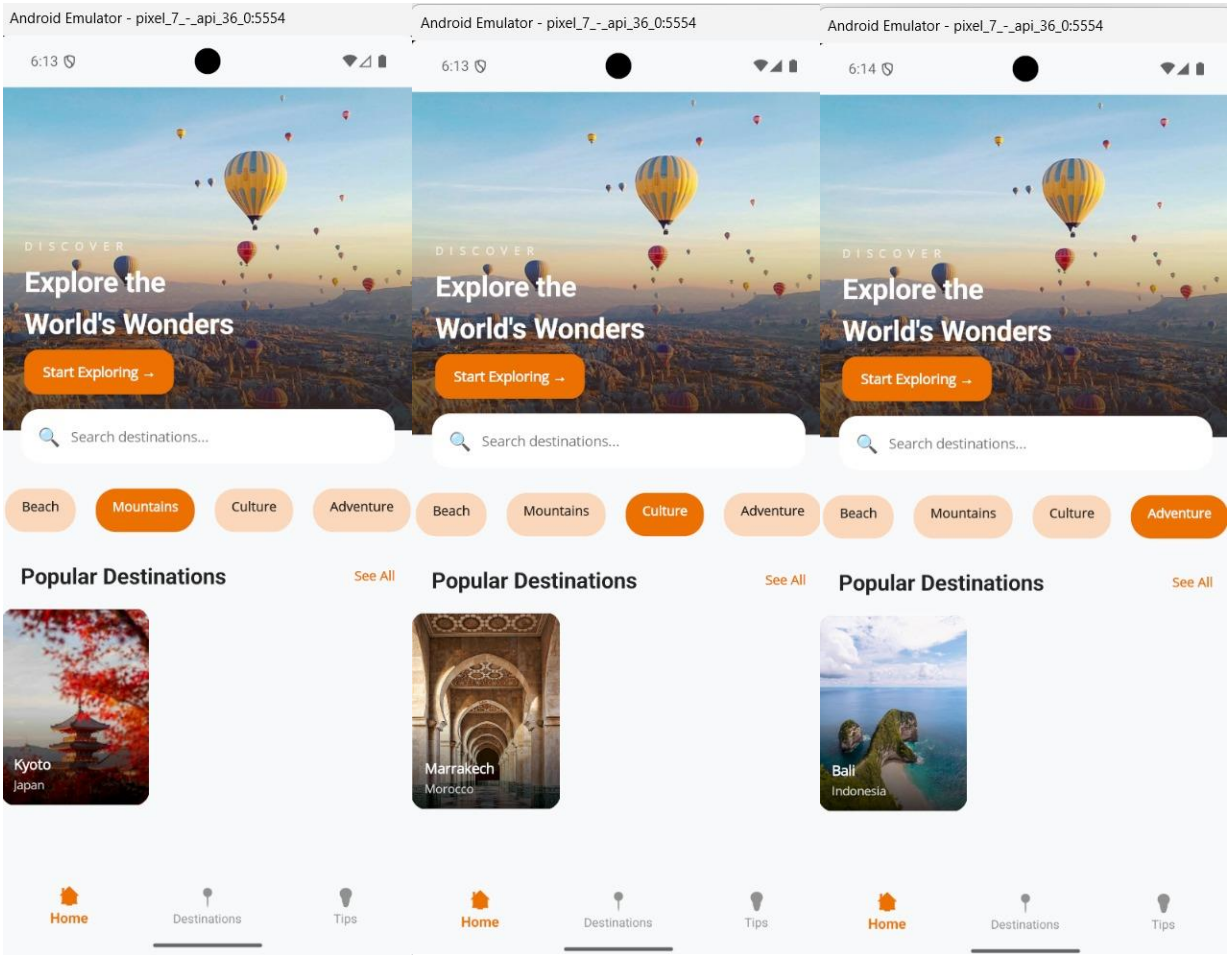

Figure 2: Category pill selection

The ApplyCategoryToPopular() private method maps each category label to a specific destination using a switch expression, then clears and re-populates the PopularDestinations collection:

```
//Popular destinations

2 references
public ObservableCollection<Destination> PopularDestinations { get; } = new()
{ };

2 references
private void ApplyCategoryToPopular(string categoryLabel)
{
    string targetName = categoryLabel switch
    {
        "Beach" => "Bali",
        "Mountains" => "Kyoto",
        "Culture" => "Marrakech",
        "Adventure" => "Bali",
        _ => "Bali",
    };

    var match = _allPopular.FirstOrDefault(d => d.Name == targetName);
    if (match is null)
        return;

    PopularDestinations.Clear();
    PopularDestinations.Add(match);
}
```

4.3 Navigation from Home Page

The Home Page provides two navigation paths. Tapping the "Start Exploring ->" button or the "See All" link navigates to the Destinations tab via Shell.Current.GoToAsync("//DestinationsPage").

Tapping a destination mini-card triggers OnDestinationCardTapped, which navigates to the Destination Detail Page for Bali or Kyoto, and shows an informational alert dialog for other destinations such as Marrakech.

```
// Kyoto and Kyoto Temples share the same detail content
bool isKyoto = key.Equals("Kyoto", StringComparison.OrdinalIgnoreCase)
    || key.Equals("Kyoto Temples", StringComparison.OrdinalIgnoreCase);

if (key.Equals("Bali", StringComparison.OrdinalIgnoreCase))
{
    TitleLabel.Text = "Bali";
    HeroImage.Source = "bali2.jpg";
    Para1Label.Text =
        "Bali is a tropical escape where emerald rice terraces, volcanic peaks, and sunlit coastlines meet a rich island culture. " +
        "From calm mornings in Ubud to golden-hour walks along the beach, the island feels both relaxing and endlessly alive.";
    Para2Label.Text =
        "Spend your days exploring waterfalls and temples, tasting local food in night markets, and unwinding in seaside cafés. " +
        "Whether you're chasing surf, nature, or quiet wellness moments, Bali offers a warm, welcoming rhythm that's easy to fall into."
    return;
}

if (isKyoto)
{
    TitleLabel.Text = "Kyoto";
    HeroImage.Source = "kyoto2.jpg";
    Para1Label.Text =
        "Kyoto is Japan's timeless heart — a city of wooden lanes, serene gardens, and temples that glow softly through changing seasons
        "It's a place where tradition is everywhere, from the scent of incense at shrines to lantern-lit evenings in historic districts.
    Para2Label.Text =
        "Wander through quiet temple grounds, pause beside koi ponds, and sip matcha in small teahouses tucked away from the crowds. " +
        "Kyoto rewards slow travel: the more gently you explore, the more beauty you'll notice in every detail.";
    return;
}
```

```
        // Fallback for other destinations
        TitleLabel.Text = key;
        HeroImage.Source = string.Empty;
        Para1Label.Text = $"{key} — details coming soon.";
        Para2Label.Text = "Explore more destinations from the Home and Destinations tabs.";
    }
```

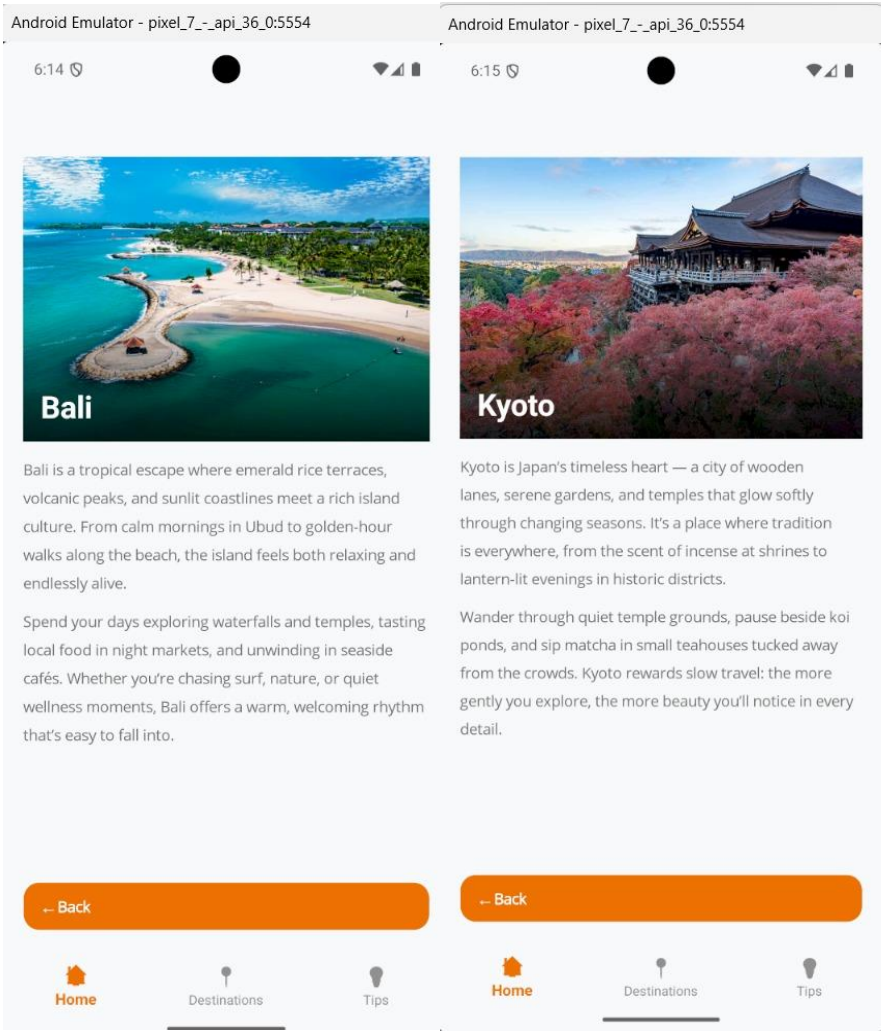

Figure 3: Destination Detailed Page when user tap Bali or Kyoto

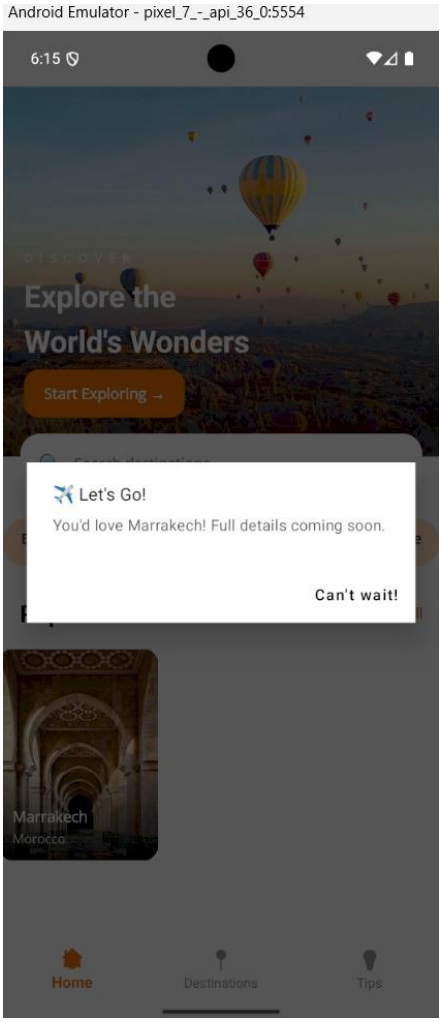

Figure 4: Alert when tapping Marrakech on Home Page

5.0 Destination Page

The second tab in this application is the Destinations Page, which is a vertically scrollable list of five destination cards, each containing rich imagery, a description, a star rating, and a heart toggle for favourites. The code for this page is in DestinationsPage.xaml, with a ViewModel in DestinationsViewModel.cs.

5.1 UI Layout and Components

The page begins with a page header "Destinations", which is in bold font and has a subtitle "Handpicked places that take your breath away". Each card in the scrollable list has:

- A full-width hero image with the destination name and location pin overlaid in white text at the bottom-left
- A heart icon button in the top-right corner of the image for toggling the favorite status.
- A short descriptive paragraph below the image.
- A star rating row showing filled (star) and unfilled (star outline) characters depending on the rating property.

The five destinations are: Kyoto Temples (Japan, 5 stars), Rice Terraces (Ubud Bali, 5 stars), Machu Picchu (Cusco Peru, 5 stars), Marrakech Souks (Morocco, 4 stars), and Patagonia Wilderness (Argentina, 5 stars).

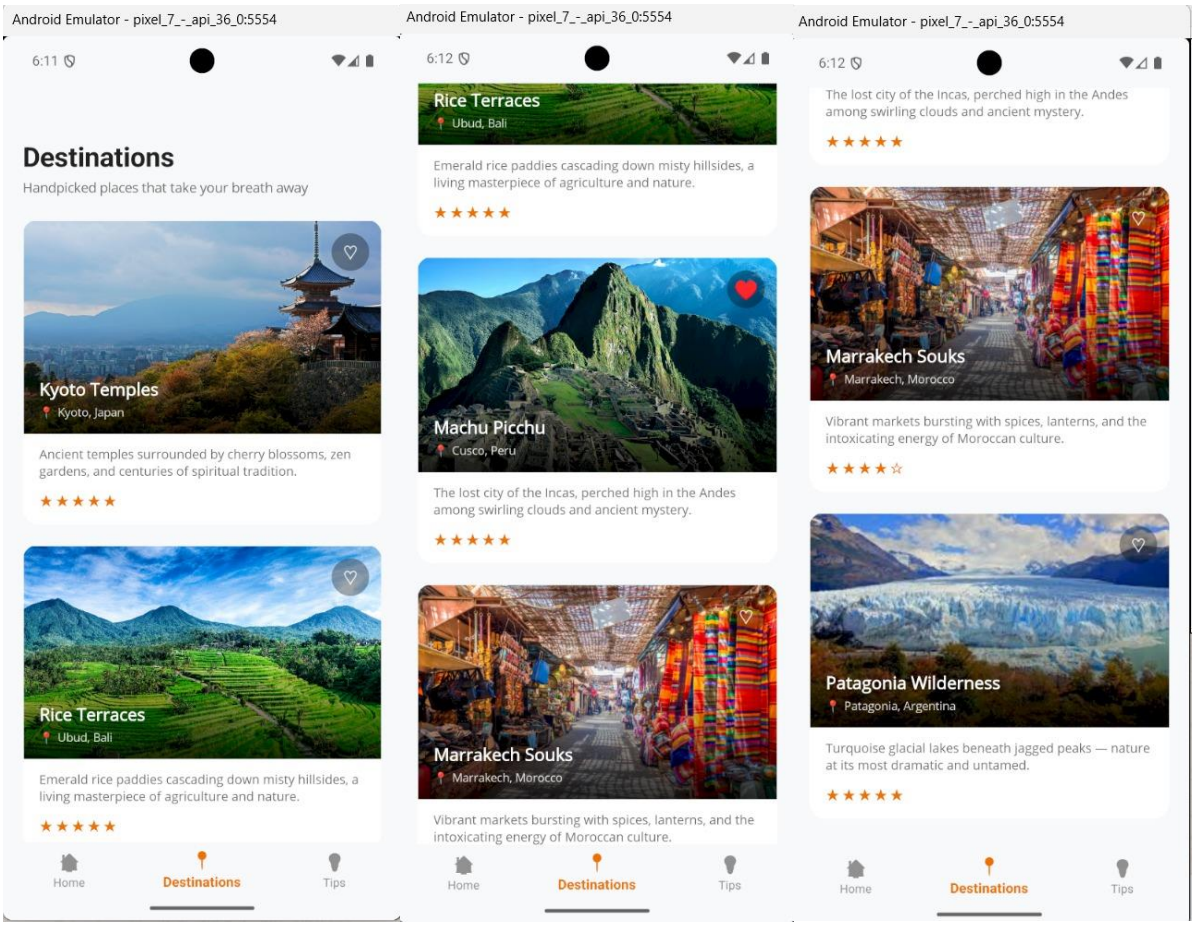

Figure 5: Destination Page

5.2 Favorite/ Heart Toggle Feature

The heart button on each destination card is used to toggle the favourite state of each destination card. This is achieved by using a ToggleFavouriteCommand in DestinationsViewModel, which simply reverses the IsFavourite state of the DestinationCardItem.

The card item class uses the “[ObservableProperty]” attribute on the isFavourite field and implements a partial method “OnIsFavouriteChanged” to send PropertyChanged events for “HeartIcon” and “HeartColor”:

```
[ObservableProperty]
private bool isFavourite;
```

```
1 reference
public string HeartIcon => IsFavourite ? "♥" : "♡";

1 reference
public Color HeartColor => IsFavourite
    ? Color.FromArgb("#D4764E")
    : Color.FromArgb("#CCFFFFFF");
```

```
partial void OnIsFavouriteChanged(bool value)
{
    OnPropertyChanged(nameof(HeartIcon));
    OnPropertyChanged(nameof(HeartColor));
}
```

If a destination is unfavorited, the heart is displayed as a transparent white outline. Once favorited, it is displayed as a filled heart in terracotta red. This gives a good visual cue to the user.

Figure 6: Machu Picchu (Favorited)

5.3 Card Tap and Alert Dialogs

Tapping a destination card will trigger the `OnCardTapped` method in the code-behind. For the Kyoto Temples, the app will go directly to the Destination Detail Page. For the other destinations, a `DisplayAlert` dialog box will appear, showing a message that the full travel guide will be available soon, with an "Exciting!" button to dismiss the dialog:

```
0 references
private async void OnCardTapped(object sender, TappedEventArgs e)
{
    var view = ViewInteraction.GestureView(sender);
    await ViewInteraction.AnimateTapAsync(view, 0.98, 85, 140);

    string name = e.Parameter?.ToString() ?? "this destination";

    //Kyoto Temples should open the Kyoto detail content with no alert messages
    if (name.Equals("Kyoto Temples", StringComparison.OrdinalIgnoreCase))
    {
        await Shell.Current.GoToAsync($"{nameof(DestinationDetailPage)}?name={Uri.EscapeDataString(name)}");
        return;
    }

    await DisplayAlertAsync("🌍 Destination Info",
        $"{name} - full travel guide coming soon!", "Exciting!");
}
```

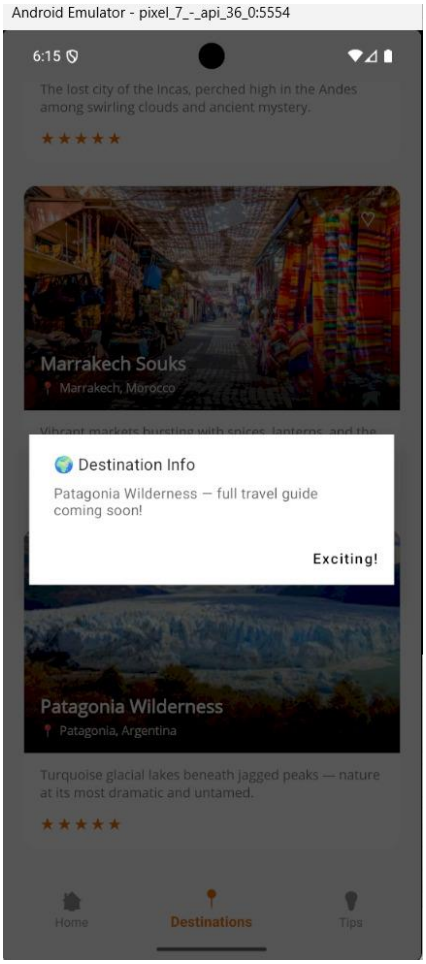

Figure 7: “Destination info” alert for Patagonia Wilderness

6.0 Travel Tips Page

The third tab in the application is the Travel Tips Page. This page is a collection of eight travel tips in a card format, each represented by a colorful emoji icon and a brief paragraph of information about the tip. The page is created in the code by the following files: TipsPage.xaml and the code-behind file named TipsViewModel.cs. At the end of the scrollable list of travel tips, a terracotta-colored banner provides a final motivational quote.

6.1 UI Layout and Components

The page header has "Travel Tips" as the title, and "Wisdom from seasoned wanderers" as the subtitle. Each of the vertical list of "Tip Cards" has:

- A colored square icon badge on the left side of the card, showing an emoji related to the category of the given piece of advice (backpack, coin bag, shield, fork and knife, speech bubble, signal bars, camera and alarm clock).
- The title of the piece of advice in bold on the right side of the icon badge.
- A muted grey descriptive sentence appearing below the title.

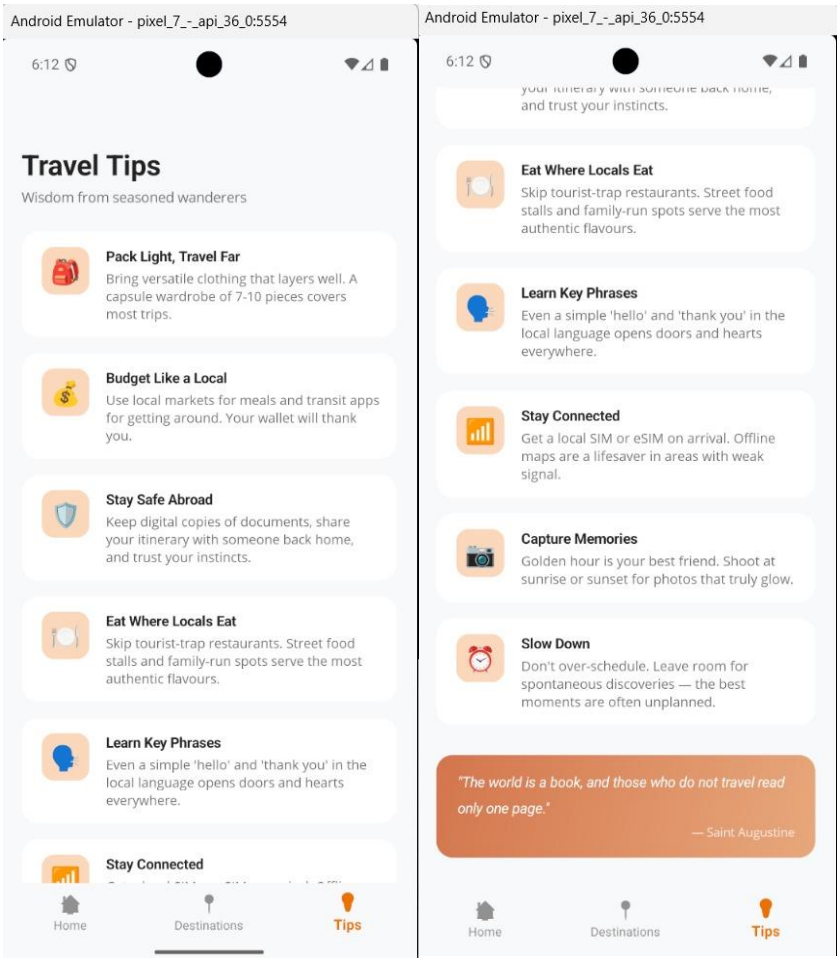

Figure 8: Travel Tips Page

6.2 Motivational Quote Banner

At the very end of the scrollable tips list, a terracotta orange-colored rounded rectangle card is shown with the following quote: "The world is a book, and those who do not travel read only one page." ~ Saint Augustine. This continues the travel theme and gives a sense of conclusion to the tips content, matching the overall color scheme of the app in the use of orange.

6.3 Tip Tap Interaction

Each of these tip cards is covered by a TapGestureRecognizer. When a user taps a tip card, the OnTipTapped method in the code-behind for the TipsPage page is invoked and executes an animation for a scale press by calling ViewInteraction.AnimateTapAsync and a DisplayAlert dialog showing a message acknowledging the specific tip title:

```
0 references
private async void OnTipTapped(object sender, TappedEventArgs e)
{
    var view = ViewInteraction.GestureView(sender);
    await ViewInteraction.AnimateTapAsync(view, 0.98, 80, 130);

    string title = e.Parameter?.ToString() ?? "this tip";
    await DisplayAlertAsync("💡 Great Tip!",
        $"\"{title}\" — bookmark this for your next adventure.", "Got it!");
}
```

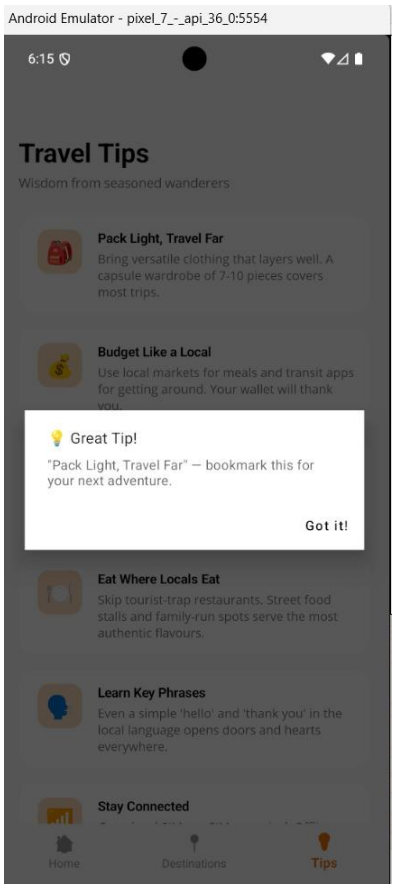

Figure 9: “Great Tip” alert on tip card top

7.0 Destination Detail Page

The Destination Detail Page offers a full-screen view of a specific travel destination. It does not have a place in the main tab bar. Instead, Shell route-based navigation is used from the Home Page and the Destinations Page. The page is defined in DestinationDetailPage.xaml and has its own code-behind file in DestinationDetailPage.xaml.cs.

7.1 Navigation and Query Parameters

The Destination Detail Page is registered as a named route in AppShell.xaml.cs during application startup:

```
    Routing.RegisterRoute(nameof(DestinationDetailPage), typeof(DestinationDetailPage));
}
```

The navigation to the page passes a name query parameter in the URI. The page has the [QueryProperty] attribute applied, which maps the "name" URL parameter to the setter of the Name property. When the setter is called, it invokes ApplyContent(name), which determines the appropriate hero image and descriptive paragraphs depending on the destination name string:

```
namespace TravelGuide.Views
{
    [QueryProperty(nameof(Name), "name")]
    18 references
    public partial class DestinationDetailPage : ContentPage
    {
        private string _name = string.Empty;

        1 reference
        public string Name
        {
            get => _name;
            set
            {
                _name = value ?? string.Empty;
                ApplyContent(_name);
            }
        }
```

7.2 Bali Detail Page

When navigated to using the name "Bali", the detail page displays a hero image of Bali (bali2.jpg), a title of "Bali", and two paragraphs of text offering a description of Bali as a tropical getaway with lush green rice terraces, volcanic highlands, and sun-kissed shores. The screenshot below illustrates this page as accessed from the Home Page popular destinations card:

Figure 10: Bali Detailed Page

7.3 Kyoto Detail Page

When the detail page is navigated to by entering the name "Kyoto" or "Kyoto Temples," the detail page will display the hero image for Kyoto (kyoto2.jpg), the title "Kyoto," and two paragraphs of text describing Kyoto as the timeless heart of Japan. The code for the function ApplyContent checks for both names by using a boolean flag isKyoto:

```
// Kyoto and Kyoto Temples share the same detail content
bool isKyoto = key.Equals("Kyoto", StringComparison.OrdinalIgnoreCase)
    || key.Equals("Kyoto Temples", StringComparison.OrdinalIgnoreCase);
```

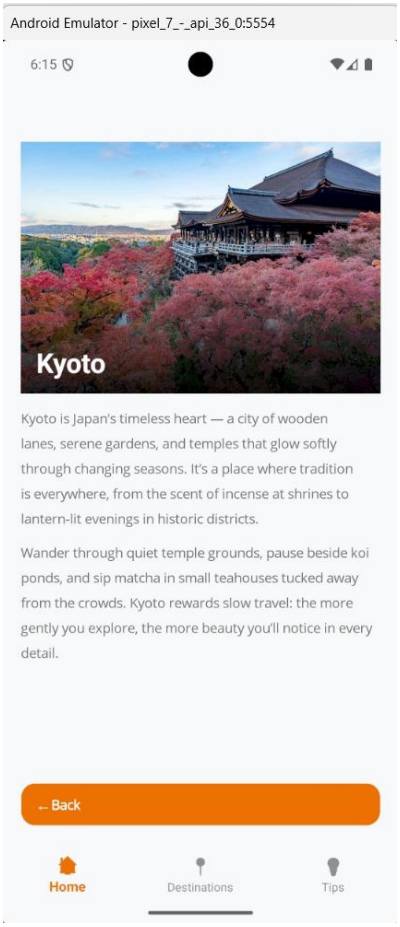

Figure 11: Kyoto Detail Page

7.4 “Back” Button Navigation

At the bottom of the detail page, there is a full-width orange-colored "Back" button. When this button is tapped, the code-behind catches the event with the "OnBackTapped" function, which plays a scale animation on the back button frame and then calls Shell.Current.GoToAsync("..") to return to the previous page in the navigation stack:

```
0 references
private async void OnBackTapped(object sender, TappedEventArgs e)
{
    await ViewInteraction.AnimateTapAsync(BackFrame, 0.96, 80, 140);
    await Shell.Current.GoToAsync("..");
}
```

8.0 Navigation Structure

TravelGuide uses .NET MAUI Shell navigation, which provides the persistent bottom tab bar and supports URI-based route navigation for sub-pages. The navigation structure is as follows:

- **AppShell.xaml:** defines the three root tab items: HomePage (//Home), DestinationsPage (//DestinationsPage), and TipsPage (//TipsPage).
- **AppShell.xaml.cs:** registers DestinationDetailPage as a named route via Routing.RegisterRoute, making it accessible via GoToAsync from any page.
- Tab navigation between Home, Destinations, and Tips uses absolute URIs such as "//DestinationsPage" to jump directly to a root page.
- Navigation to the Detail Page uses relative URIs with query parameters such as "DestinationDetailPage?name=Bali", and the Back button uses the ".." relative URI to pop the navigation stack.

The tab icons change to terracotta orange when the tab is selected and grey for the unselected ones. This provides the user with direct feedback about the current position within the application. The text for the selected tab is also painted in orange, while the texts for the unselected ones remain grey. This is true for all the three main tabs, as validated by the screenshots collected during the testing.

9.0 Conclusion

TravelGuide is an effective example that proves the potential of the .NET MAUI framework as an innovative cross-platform mobile application development framework. The application is well-equipped with an excellent set of features that make it an effective travel guide application with four pages, including an interesting Home Page with hero images, an information-packed Destinations Page with interactive favouriting, an informative Travel Tips Page with information that can be interacted with by tapping on the screen, and an interesting Destination Detail Page with route-based navigation for an enhanced experience.

The MVVM design pattern with CommunityToolkit.Mvvm is used to maintain the separation of concerns, thus ensuring the maintainability and testability of the code. The ViewInteraction class is used as a central location to handle the application's animation, ensuring consistency throughout the application while avoiding code duplication. The terracotta and cream color scheme, along with the authentic travel images used, makes the application visually appealing and gives the application a professional look.

As discussed in the laboratory requirements, the application is well-equipped with the following features: an interesting Home Page with a tagline, images, and a welcome message; two well-equipped secondary pages with relevant information; smooth navigation between all the pages; an interesting and innovative design; and various interactive features such as animations, alerts, route navigation, real-time data binding, etc.

The application can be further enhanced with the following features: the implementation of the search bar with real-time filter functionality; the implementation of the Destination Detail Page with all the destinations; the implementation of user authentication to save favourites; the implementation of the live travel API to provide real-time information on destinations.

10.0 References

- Microsoft. (2024). .NET MAUI documentation. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/maui/
- Microsoft. (2024). Shell navigation in .NET MAUI. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/maui/fundamentals/shell/navigation
- Microsoft. (2024). Shell navigation in .NET MAUI. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/maui/fundamentals/shell/navigation
- CommunityToolkit. (2024). MVVM Toolkit documentation. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/
- Microsoft. (2024). Data binding and MVVM in .NET MAUI. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/maui/xaml/fundamentals/mvvm
- Microsoft. (2024). Animations in .NET MAUI. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/maui/user-interface/animation/basic
- Microsoft. (2024). Gesture recognizers in .NET MAUI. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/maui/fundamentals/gestures/
- Microsoft. (2024). ObservableCollection in .NET. Microsoft Learn. https://learn.microsoft.com/en-us/dotnet/api/system.collections.objectmodel.observablecollection-1